

Mindweave

A Development Journal

What we built, why we built it that way, what broke, and how it got fixed — written to be read by anyone, technical or not.

Prepared for Gayathri Rajeev · September 2026

How to read this document

This is a full, honest account of how Mindweave grew from a working note-taking app into the system it is today — every feature, every decision, and every mistake, in the order it actually happened. It was written for two different readers at once, so both can follow the same document:

- **If you're not technical:** read the *"In plain terms"* lines in every section. They explain what changed and why in everyday language, with no assumed background.
- **If you're technical:** the "How it works, technically" and "What went wrong, and the fix" sections underneath give the real implementation detail — file names, root causes, and the actual reasoning behind each design choice, not just the end result.

The document follows the same order the work actually happened in, including the false starts and corrections — because the reasoning behind a decision usually only makes sense next to the problem that provoked it.

Part 1 — What Mindweave Is

***In plain terms.** Mindweave is a personal knowledge base. You throw things at it — a link, a typed note, a voice memo, a photo of a receipt, a research paper, a YouTube video — and it remembers all of it, understands it, and lets you ask questions about it later in plain English, the way you'd ask a person who'd actually read everything you saved.*

How it works, technically. The backend is a Python FastAPI service; the frontend is a React (Vite) single-page app. Saved items live in Postgres (hosted on Supabase), with pgvector storing an embedding for every chunk of text so questions can be answered by semantic similarity search rather than keyword matching. All AI capability — embeddings, chat, transcription, image generation, text-to-speech, video understanding — runs through a single Gemini API key, deliberately funneled through one module (`gemini_client.py`) so the whole surface area Google actually sees is auditable in one place.

Why we built it this way. Before any of the work in this journal began, Mindweave already did the core job: save a URL or a note, chunk and embed it, answer questions over it with retrieval-augmented generation (RAG), and automatically extract concepts to build a graph of how ideas connect — with real Postgres Row-Level Security enforced so one user's data is structurally unreachable by another, not just filtered out by application code that could have a bug in it. Everything documented below was added on top of that foundation, one real feature at a time.

Part 2 — The Build Log

Each entry below is a real step in the project, in the order it happened. Some were requested outright; some emerged from a bug found while testing something else; a few were corrections to work done earlier in the same session.

1. Voice notes that keep their own language

***In plain terms.** You can record or upload a voice note in any language, and Mindweave writes it down in that same language instead of quietly translating it to English.*

Why we built it this way. The transcription model had no explicit instruction either way, so there was a real risk it would translate rather than transcribe — a subtle failure that could go unnoticed for a long time since the output still looks like a normal transcript, just not in the language actually spoken.

How it works, technically. A system instruction was added telling the model to detect the spoken language and write the transcript in that language, without translating — even mid-recording, if someone switches languages.

What went wrong, and the fix. The very first attempt crashed outright: the dedicated speech-to-text model rejects any request that includes a system instruction at all ("Developer instruction is not enabled for this model"). The fix was to drop the instruction in as a plain text part alongside the audio in the input list instead of as a system instruction — a distinction this particular model turned out to enforce strictly.

2. Upload a document or receipt, get structure back

***In plain terms.** You can upload a PDF or a photo — a receipt, a research paper, a form — and instead of just storing it as an unreadable blob, Mindweave reads it and pulls out the useful parts: a receipt's line items and total, a paper's figures, a form's key fields.*

How it works, technically. One structured-output call classifies the document type and returns full text, key fields, line items, and figure descriptions in a single pass, using whichever of these apply — a plain note gets full text with everything else empty, a receipt gets line items and totals, a paper gets figures. The full text is what gets embedded and made searchable; the structured extras are stored alongside for display.

What went wrong, and the fix. The API distinguishes a "document" content block (PDF only) from an "image" one (JPEG/PNG/WEBP/HEIC) — the first working version tagged every upload as a document regardless of file type, which would have rejected every image upload the moment someone actually tried one. Caught and fixed before it shipped, by branching on the actual MIME type.

3. Making the upload button findable, and letting it take several files

***In plain terms.** The document-upload option was buried two clicks deep in a small "+" menu, easy to miss entirely. It got its own visible button, and you can now select several files at once instead of one at a time.*

Why we built it this way. Discoverability was a real usability problem, not a hypothetical one — it came directly from trying to use the feature and not being able to find it.

4. Asking questions by voice, not just typing

***In plain terms.** The chat box now has a microphone button: speak your question, and it's transcribed and asked automatically, the same way typing and pressing enter would.*

How it works, technically. A separate endpoint transcribes audio without saving it as an item — distinct from the existing save-a-voice-note flow, since asking a question shouldn't create a permanent note.

5. The assistant can draw you a picture when that helps

***In plain terms.** If a question calls for a diagram or illustration rather than a wall of text, the assistant can generate one inline as part of its answer.*

How it works, technically. The chat model's structured response includes an optional image prompt field; it's asked to fill that in only when an image would genuinely help, then a second Gemini call renders it. If image generation fails for any reason, the text answer still comes back — a picture is a bonus, never a dependency.

Why we built it this way. Given the deliberate choice to let the model decide per-question rather than requiring an explicit "draw me a picture" trigger phrase, it will occasionally generate an image when one wasn't really wanted, or skip one that would have helped — a known and accepted trade-off of that approach.

6. A command palette (Ctrl+K)

***In plain terms.** Press Ctrl+K (or Cmd+K on a Mac) anywhere in the app to get a search box: type to find any saved item, or jump straight to an action like starting a recording or asking a question, without touching the mouse.*

How it works, technically. Triggering an action from the palette (like focusing the note box, or scrolling to a specific item) required a way for one component to reach into another. React's `forwardRef/useImperativeHandle` pattern was introduced here for the first time in the codebase specifically to make that possible, and every later feature that needed the same kind of cross-component jump reused it rather than inventing a new mechanism.

7. Three views of the same saved items

***In plain terms.** The same saved items can now be browsed as a card list (the original view), a sortable table, or a month calendar showing which days you saved something.*

Why we built it this way. This idea came directly from looking at two competing note-taking products for inspiration. One of them (Mind Weaver) markets itself around exactly this principle — "same data, every lens" — letting the same underlying items be viewed as a mindmap, kanban board, table, or calendar. It was judged the single most valuable idea worth adopting, versus more surface-level gimmicks (an animated mascot) that weren't worth the effort.

How it works, technically. All three views read the exact same underlying item list — no new API calls, no schema changes. Jumping to a specific item from elsewhere in the app (the command palette, a citation) forces the view back to the plain list first, since that's the one view guaranteed to have every item rendered and addressable in the page.

8. A daily reminder of something you saved a while ago

***In plain terms.** Once a day, a small card appears showing one older saved item — something saved more than two days ago — so the app occasionally reminds you of things instead of only ever being a one-way archive you write into and never revisit.*

How it works, technically. Entirely client-side: which item to show is picked by a small deterministic hash of the current date, so it's the same pick all day and across browser tabs, but changes the next day — no server-side scheduling needed for something this low-stakes. Dismissing it is remembered in the browser's local storage for that day only.

9. Chat answers cite exactly where they came from

***In plain terms.** Every answer that drew on your saved items now shows small clickable tags underneath naming the source; clicking one jumps straight to that item.*

What went wrong, and the fix. This closed a real gap: the backend had been quietly returning which items an answer used since the RAG system was first built, but nothing in the interface ever displayed it — it was pure dead data, visible only in a browser's network inspector. Wiring it up meant changing what the field actually carried, from a bare list of item IDs to a list of {id, title} pairs, so the frontend never has to do a second lookup just to show a label.

10. Understanding YouTube videos, not just scraping their page

***In plain terms.** Save a YouTube link, and instead of getting the mostly-useless cookie-banner text you'd get from scraping the page's HTML, Mindweave actually watches the video — what's shown on screen as well as what's said — and writes a real summary.*

How it works, technically. Gemini accepts a video by URL directly as input, with no upload step, and produces a title, a detailed summary (which becomes the item's searchable text, the same role a document's extracted text plays), and a handful of key points. If that call fails for any reason — an

age-restricted video, a quota limit, anything — the code quietly falls back to the ordinary page-scrape every other URL gets, so a save never hard-fails outright.

What went wrong, and the fix. The very first real attempt appeared to fail: the saved item came back with scraped page boilerplate instead of a summary, and the request took an unusually long time before landing. Calling the underlying function directly, outside the running server, showed it actually worked perfectly on retry — the original failure was a transient rate-limit or timeout, not a bug in the code. The lesson carried forward from here: verify with a live, isolated call before assuming a fix is needed.

11. Better answers as the library grows, at no extra cost

In plain terms. Chat answers get more accurate as you save more, instead of gradually getting muddier as unrelated-but-similar-sounding notes start competing for attention.

Why we built it this way. Retrieval had been picking a small fixed number of matches purely by how mathematically similar they were to the question — fine with a handful of saved items, but with no way to recover if a merely similar-sounding match crowded out a genuinely relevant one as the collection grew.

How it works, technically. Two-stage retrieval, folded into a single Gemini call rather than two: pull a much wider set of candidate excerpts by similarity (cheap — it's just a vector index scan), number them in the prompt, and have the model report back exactly which numbered excerpts it actually used to answer. Citations are then built only from what was genuinely used, which happens to make citations more precise as a side effect of the same change. Deliberately built as one call instead of the more conventional two-call rerank-then-answer pipeline, specifically to avoid doubling the API cost per question given how tightly the Gemini free tier was already being felt.

12. A detour: fixing who commits said they were

In plain terms. For a stretch of commits, GitHub was showing the wrong person as the one who "committed" the changes (a leftover name from a different account configured on the same machine), even though the actual author was correct. It was corrected, and the machine's settings were fixed so it can't happen again.

How it works, technically. Git actually tracks two separate identities per commit: the *author* and the *committer*. Passing `--author` on the command line only overrides the first — the committer identity still comes from the local machine's git configuration. Five already-pushed commits needed their history rewritten (an interactive rebase re-applying each commit under the correct committer identity) and force-pushed, after explicit confirmation since rewriting shared history is exactly the kind of action that warrants asking first. The machine's global git config was then corrected, so every future commit gets both identities right automatically.

13. A strategy conversation: what actually impresses a recruiter

***In plain terms.** Asked how to make the project attract recruiters, the honest answer given was: don't chase unfamiliar trendy technology for its own sake — the stack was already legitimate and current. What actually matters is a live demo people can click into, a README that explains the real engineering decisions, genuine test coverage, and one differentiating capability built well rather than several built shallowly.*

Why we built it this way. This reasoning shaped everything that came after it in this journal: the decision to build backend tests, to write this very document, and to plan a real deployment all trace back to this conversation rather than being requested individually from scratch.

14. A detour: checking a claim before building on it

***In plain terms.** A request came in to use "Higgsfield," a video-generation service, on the understanding that its API was now free. Before writing any code against it, that claim was checked directly — and it turned out to be false. Higgsfield's consumer app has a limited free tier, but its API is billed per second of generated video (a few cents per second, so roughly seventy cents to just over a dollar for a ten-second clip), with only a 24-hour trial.*

Why we built it this way. Building against an unverified claim about a third-party API's pricing would have either wasted the effort once the real cost surfaced, or quietly introduced a second paid AI provider into a system explicitly designed around a single one. Verifying first, in public with sources cited, avoided both outcomes and let the actual decision — skip it, in favor of a cheaper approach built entirely on the existing Gemini key — be made with real information instead of a second-hand claim.

15. Folders — the biggest single feature

***In plain terms.** You can now create named folders (like ChatGPT or Claude "Projects") and file any kind of saved item into one. Each folder gets its own chat, scoped to only what's inside it, and that chat now actually survives a page refresh — which, notably, no chat in the app had ever done before this feature. Deleting a folder deletes everything filed inside it, the same way deleting a folder on a computer takes its contents with it. The main chat that searches everything you've ever saved is untouched and still there alongside folders.*

Why we built it this way. This was scoped carefully before any code was written, because two of the natural-sounding designs conflicted with each other once examined closely: "an item can belong to several folders at once" and "deleting a folder deletes its items" cannot both be true at the same time without a confusing edge case (deleting one folder silently destroying an item that also still logically belonged to a second, untouched folder). Once that tension was pointed out directly, the simpler, more predictable model — one folder per item, exactly like a real filesystem folder — was chosen deliberately over the more flexible one.

How it works, technically. A new folders table, a nullable folder_id column on every saved item, and (for the first time) a chat_messages table so conversation history is actually persisted rather than living only in the browser's memory. Every endpoint that lists items or asks a question accepts an optional folder scope. On the frontend, a folder bar sits above the three main panels; picking a folder scopes

both the item list and the chat beneath it simultaneously, since they share the same piece of state one level up in the component tree.

What went wrong, and the fix. Jumping to a specific item from the command palette, a citation, or the daily resurfacing card could silently fail to find it on screen if that item happened to live inside a folder other than the one currently open — the item genuinely existed, just not in the currently-loaded list. Fixed by having every one of those jump actions force navigation back to the unfiled "Home" view first, since Home is the one place guaranteed to show every item regardless of which folder it's filed in, and by properly waiting for that view to finish reloading its data before trying to scroll to the item — an earlier version of the fix used a fixed short delay that worked when already on Home but was too short to survive an actual folder switch, which reloads data from the server and takes longer.

16. Real tests for the folders feature

***In plain terms.** Thirty automated checks now run against the folders, item-filing, and chat-history code, up from nine that existed before — covering things like "does deleting a folder actually delete everything in it," "can one person's folder be seen or touched by someone else's account," and "does a folder's chat history stay completely separate from another folder's."*

Why we built it this way. This project already had a stated rule for its existing tests: prefer a real database over faking one, because faking the database mostly ends up testing the fake, not the actual code. There was no test database in place yet to honor that rule for anything new, so one was built rather than quietly breaking the project's own standard for convenience.

How it works, technically. A disposable, local Postgres database (with the pgvector extension, matching production) runs in Docker, completely separate from the real Supabase database, with a small hand-built stand-in for just the parts of Supabase's authentication system the database schema depends on. The tests skip themselves cleanly — rather than fail — on any machine where that database isn't running, so the rest of the test suite stays usable everywhere.

What went wrong, and the fix. Two real bugs surfaced while building this test setup, not in the app itself but in the tests meant to check it. First, reusing one database connection across multiple tests threw a confusing "another operation is in progress" error — the testing framework hands each test its own independent event loop, and a database connection created for one loop cannot safely be reused on a different one; the fix was a fresh connection per test rather than one shared connection. Second, checking whether the test database was even reachable on every single test made the whole suite take over a minute to simply confirm the database was turned off; the fix was to check exactly once per run and remember the answer, dropping that same check down to about six seconds.

17. Fixing the concept graph looking cut off

***In plain terms.** The concept-graph panel on the right could end up rendering into a smaller area than it actually had room for, making it look clipped or cut in half. It's fixed now, and the panel is bigger by default too.*

How it works, technically. The graph's canvas had only been resizing itself in response to the browser window changing size, which misses every other reason its own container can resize — dragging the panel's own resize handle, or the page's layout settling right after it first loads. Switched to a `ResizeObserver`, which watches the actual container element instead of the window, so it catches every case correctly.

18. A quietly broken button, caught before it mattered

In plain terms. While reviewing the backend for an unrelated reason, a real bug was found: moving an item into a folder was using a network request method the server's cross-origin security settings didn't actually permit. In a real browser this specific action would have been silently rejected; it had simply never been tested against the app running in an actual browser tab at the point it was introduced.

How it works, technically. The server's list of allowed request methods was missing `PATCH`, the method the move-to-folder endpoint uses. Command-line testing tools like `curl` don't enforce this kind of browser security check at all, which is exactly why earlier spot-checks with `curl` hadn't caught it. One line added to the allowed-methods list fixed it.

19. Explain any item as a narrated slideshow

In plain terms. Click the film-strip button on any saved item, and Mindweave turns it into a short narrated explainer: a handful of scenes, each with a generated picture and a spoken narration, playable right in the app like a simple slideshow with a voice-over.

Why we built it this way. This began as a request to use a third-party video-generation service, which the check described in entry 14 ruled out on cost and architecture grounds. The actual underlying want — being able to get a video-like explanation of a saved item on request — was still worth building; it just needed a version that didn't require a second, per-second-billed AI provider. Gemini turned out to already support both the image generation and the text-to-speech pieces needed to build exactly that.

How it works, technically. One call generates a script broken into three to five scenes, each with a line of narration and a separate description of what image should accompany it; each scene's image and narration audio are then generated separately. If one scene's image or audio generation happens to fail, that scene still appears with whatever it does have rather than the whole explainer failing outright. Given each request can cost up to eleven separate Gemini calls, this endpoint carries its own much tighter rate limit than the rest of the app.

Part 3 — What Kept Coming Up

A few patterns repeated often enough across the entries above that they're worth naming on their own, separate from any one feature.

Verify before building, especially against someone else's claim

Twice in this journal, a request rested on a claim that turned out not to hold up once actually checked — that a third-party API was free, and (implicitly, before it was tested) that a video-generation model was reachable through the existing Gemini setup at all. In both cases, checking first rather than building first avoided wasted work and, in the Higgsfield case, avoided quietly taking on a real ongoing cost.

A single AI provider, on purpose

Every AI capability in Mindweave — text generation, embeddings, transcription, image generation, text-to-speech, video understanding — runs through one Gemini API key, funneled through one file in the codebase. This was treated as a real constraint worth protecting, not just an accident of how the project started: it came up explicitly when a request would have introduced a second paid provider, and the cheaper, single-provider alternative was chosen instead.

Cost-consciousness shaped real technical decisions

Running into the Gemini free tier's daily request limit repeatedly during ordinary testing wasn't just an inconvenience — it directly shaped how later features were designed: reranking was deliberately built as one Gemini call instead of the more conventional two, and the explainer feature's rate limit was set specifically because each use is unusually expensive compared to a normal chat turn.

Real bugs surfaced by real testing, not by assumption

Several genuine bugs in this journal were only found by actually exercising the code — calling a function directly, checking a live server's registered routes, running against a real browser's security rules instead of curl — rather than by reasoning about the code in the abstract. The CORS/PATCH bug in particular would have been invisible to every check performed except an actual browser request.

Ask before the expensive or hard-to-reverse decisions, not before every decision

Clarifying questions were asked at genuine forks in the road — whether an item can live in several folders at once, whether deleting a folder should delete its contents, which cloud hosts to use — and skipped for the many smaller implementation choices in between, which were just made and explained afterward. Rewriting already-pushed git history was treated as exactly the kind of action that needed explicit confirmation first, given it cannot be undone once force-pushed.

Part 4 — Where Things Stand, and What's Next

Shipped and working

- Multi-language voice transcription for saved notes and spoken chat questions
- Document/receipt/PDF upload with structured data extraction
- YouTube video understanding, with an automatic fallback if it fails
- On-demand generated images inside chat answers
- A command palette (Ctrl+K) for keyboard-driven navigation and actions
- Three interchangeable views of saved items: list, table, calendar
- Type filtering (notes / links / videos / audio / documents)
- Daily resurfacing of an older saved item
- Clickable citation chips on every chat answer that used saved items
- Single-call retrieval reranking for more relevant answers as the library grows
- Folders with persisted, scoped chat and cascading delete
- Thirty automated integration tests covering the folders/chat feature set
- A fixed, larger concept-graph panel
- On-demand narrated explainer slideshows for any saved item

Deliberately not built, and why

- **Third-party video generation** — checked and found to be a paid, per-second-billed service that would have introduced a second AI provider; replaced with the Gemini-only narrated explainer instead.
- **A live, back-and-forth conversational voice mode** (speaking with the assistant continuously, the way a phone call works, rather than recording one question at a time) — would need a genuinely different, streaming connection to the AI model than the current record-then-transcribe flow, so it was deliberately deferred as its own piece of work rather than folded into the smaller voice-input feature that shipped.

Known, deliberately deferred

- Deployment: nothing is live on the public internet yet; a specific plan (Render for the backend, Vercel for the frontend) was chosen but not yet executed at the time of this document.
- The Gemini API key is still on the free tier, which is what has caused most of the rate-limit interruptions described in this journal; upgrading it is a decision only the account owner can make.

- A global, app-level daily spend or request cap, so a single very active session can't exhaust the day's quota for everyone — discussed as a real need once the app is actually public, not yet built.

End of journal. This document reflects the project's state as of the date on the cover page; Mindweave has almost certainly kept moving since.